Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 03 - VHDL - Hardware-Beschreibungssprache

Prof. Dr.-Ing. Guillermo Payá Vayá, 26. November 2024

Simulation und Testbench

Simulation

- Eine verhaltensorientierte VHDL-Beschreibung lässt sich mit einem Simulationswerkzeug kompilieren und relativ effizient simulieren
- Um ein in VHDL beschriebenes System simulieren zu können, ist eine sog. *Testbench* erforderlich, die das zu testende System instanziiert und mit geeigneten Testvektoren versorgt
  - Einige FPGA-Entwicklungsumgebungen erstellen automatisch die Testbench als VHDL-Datei, die bereits die Instanziierung des DUT (*Design under Test*) und evtl. die Takterzeugung beinhaltet
- Die Simulation kann auf unterschiedlicher Art durchgeführt werden
  - Nur die *Funktionalität* der Beschreibung wird überprüft, ohne auf die technologiespezifische Implementierung zu achten
  - Die *Timing*-Eigenschaften der fertig synthetisierten Schaltung werden in die Simulation einbezogen $\rightarrow$ zeitaufwändiger

Zeitliches Verhalten: `wait`-Anweisungen

- Die Ausführung des Prozesses wird unterbrochen bis die in der `wait`-Anweisung angegebene Bedingung erfüllt wird
- Arten der `wait`-Anweisungen
  - Warten bis eine gewisse System-Zeit vorüber ist (`wait for`)
  - Warten bis ein bestimmtes Ereignis eintritt (`wait until`)
  - Warten bis sich ein Signal ändert (`wait on`)
    - Eine `wait on` Anweisung mit mehreren Signalen am Anfang des Prozesses entspricht dem Verhalten einer Sensitivity-List
- Die meisten `wait`-Anweisungen sind gar nicht (`wait for`) oder nur sehr schwer synthetisierbar. Daher sollten sie deshalb **nur** für Simulationsmodelle oder Testbenches verwendet werden.
- Unterschiedliche `wait`-Anweisungensarten kann man konkatenieren

```
wait until signal_trigger for 1 ms;
```

Zeitliches Verhalten: `wait`-Anweisungen

```
rst_gen : process  -- Leere Sensitivity-List (sofort ausführen)
begin
  reset <= '1';    -- Aktivieren den Reset
  wait for 100 ns; -- Warte bis 100ns der Simulation vergangen sind
  reset <= '0';    -- Deaktiviere den Reset
  wait;            -- Warte unendlich
end process rst_gen;
```

Zeitliches Verhalten: `wait`-Anweisungen

```
rst_gen : process  -- Leere Sensitivity-List (sofort ausführen)
begin
  reset <= '1';     -- Aktivieren den Reset
  wait for 100 ns; -- Warte bis 100ns der Simulation vergangen sind
  reset <= '0';     -- Deaktiviere den Reset
  wait;             -- Warte unendlich
end process rst_gen;
```

```
process  -- Leere Sensitivity-List (sofort ausführen)
begin
  result <= a and b;
  wait on a, b; -- Warte bis a oder b ändert
end process;
```

Zeitliches Verhalten: `wait`-Anweisungen

```
rst_gen : process  -- Leere Sensitivity-List (sofort ausführen)
begin
  reset <= '1';     -- Aktivieren den Reset
  wait for 100 ns; -- Warte bis 100ns der Simulation vergangen sind
  reset <= '0';     -- Deaktiviere den Reset
  wait;             -- Warte unendlich
end process rst_gen;
```

```
process  -- Leere Sensitivity-List (sofort ausführen)
begin
  result <= a and b;
  wait on a, b; -- Warte bis a oder b ändert
end process;
```

```
process  -- Leere Sensitivity-List (sofort ausführen)
begin
  result <= a and b;
  wait until trigger; -- Warte bis signal trigger ist '1'
end process;
```

wait-Anweisungen

Zeitliches Verhalten: `after`-Zuweisungen

- `after` kann verwendet werden, um anzuzeigen, zu welchem Zeitpunkt einem Signal ein Wert zugewiesen wird

```
signal_name <= init_value,
               value_1 after time_1,
               value_2 after time_2,
               ... ;
```

- Beispiel:

```
reset <= '1', '0' after 100 ns;
```

```
clk_gen : process  -- Leere Sensitivity-List (sofort ausführen)
begin
  clk <= '0', '1' after 100 ns;
  wait for 100 ns;
```

after-Anweisungen

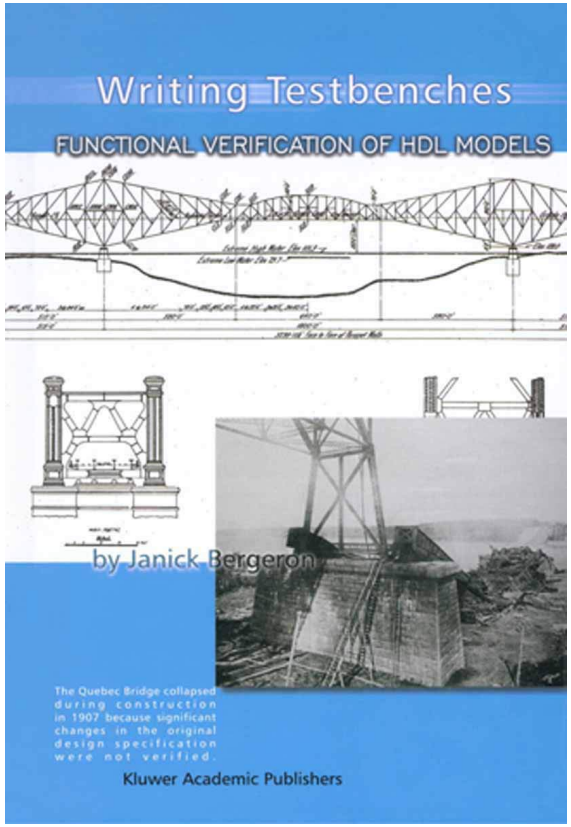
Testbench (I)

- Eigenschaften einer VHDL-Testbench:
  - Die Entity hat keine Ein- oder Ausgänge
  - In der Architektur werden das zu verifizierende (Sub-)System und sonstige erforderliche Modelle (z.B. Speichermodelle) instanziiert
  - Der VHDL-Code einer Testbench muss sich nicht an irgendwelche synthesespezifische Beschreibungsrichtlinien halten
    - File I/O und print-Ausgaben sind möglich (`package testio`)
    - `wait`-Anweisungen und `after`-Zuweisungen, in der die Zeitpunkte der Signalübergänge festgelegt sind, dürfen verwendet werden

```
entity DESIGN_TB is
end DESIGN_TB;
architecture TESTBENCH of DESIGN_TB is
  -- Schnittstellensignale zu untersuchenden entity
begin
  -- Signalzuweisungen an die Stimulisignale
  -- mit Angabe des Zeitpunkts der Signalübergänge
  bezeichner : DUT
    generic map ( ..)
    port map ( ... );
end architecture TESTBENCH;
```

Testbench (II)

- Grundlegende Verifikationsstrategien
  - Selbstcheckende verhaltensorientierte Testbench-Beschreibung
  - Testvektoren: externes Programm erstellt eine Datei mit Eingangs- und erwarteten Ausgangsdaten, die in der Simulation als Referenzdaten verwendet werden
- Für komplexe Designs werden spezifische Hardware-Verifikationssprachen (*Hardware Verification Lenguages (HVL)*) verwendet, z.B. SystemC oder SystemVerilog
  $\rightarrow$ Neue Vorlesung in Sommersemenster: **"Softwaremodellierung elektronischer Systeme"**



Assertions: `assert`-Anweisungen

- `assert`-Anweisungen erkennen Bedingungen, die immer wahr sein sollten, d. h. sie prüfen, ob die erwarteten Bedingungen im Modell erfüllt sind.
- Eine `assert`-Anweisung ist eine sequentielle Anweisung, so dass sie an jeder beliebigen Stelle im Textkörper eines `process` eingefügt werden kann.
- `assert`-Anweisungen werden bei der Synthese ignoriert.
- Syntax:

```
label: assert bedingung report "text" severity severity_label;
```

  - Wenn die Bedingung während der Simulation nicht erfüllt ist, printet der Simulator den Assert-Text.
  - Das `severity_label` gibt den Grad der Violation an. Mögliche Label sind `note`, `warning`, `error(default)`, und `failure`.

Assertions: `assert`-Anweisungen (Beispiel I)

```
entity max3 is
  port (a, b, c: in integer; z : out integer );
end entity max3;
architecture check_error of max3 is
begin
  maximizer : process (a, b, c)
    variable result : integer;
  begin
    if a > b then
      if a > c then
        result := a;
      else
        result := a;  -- Oops! Should be: result := c;
      end if;
    elsif b > c then
      result := b;
    else
      result := c;
    end if;
    assert result >= a and result >= b and result >= c
      report "Inconsistent result for maximum: " & to_string(result)
      severity failure;
      z <= result;
  end process maximizer;
end architecture check_error;
```

assert-Anweisungen 1

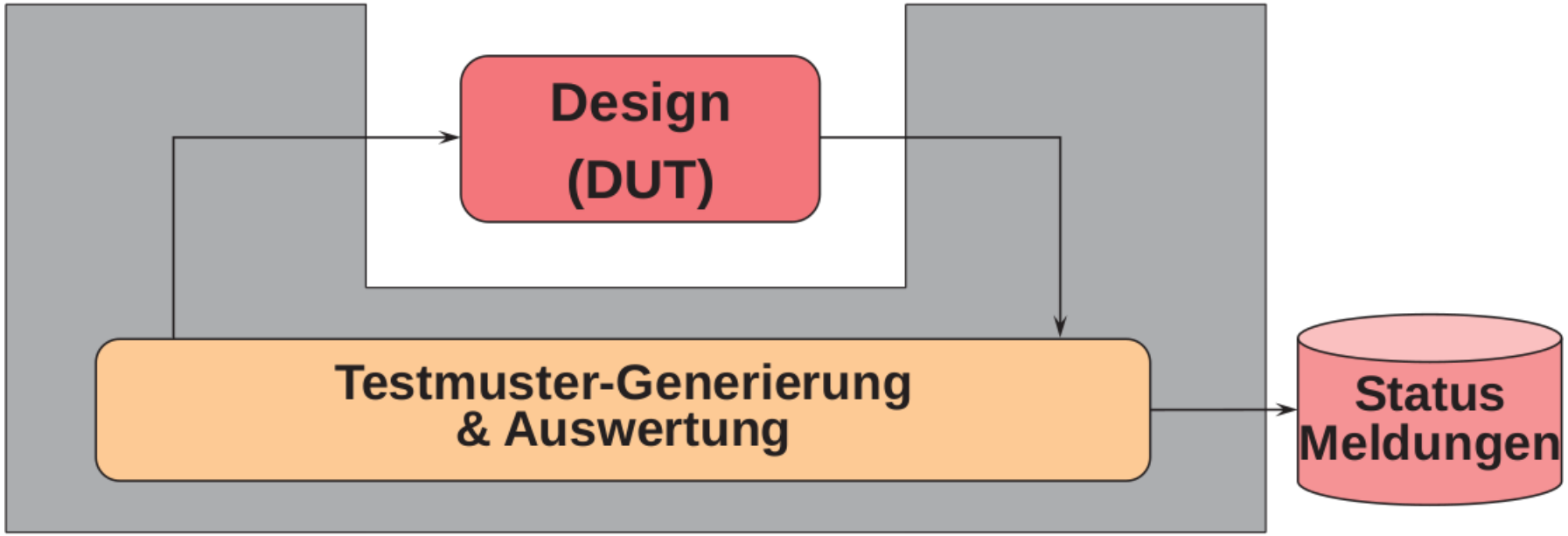
Assertions: `assert`-Anweisungen (Beispiel II)

```
entity edge_triggered_register is
port (clk   : in std_ulogic;
      d_in  : in real;
     d_out  : out real);
end entity edge_triggered_register;
architecture check_timing of edge_triggered_register is
begin
  store_and_check : process (clk) is
    variable stored_value : real;
    variable pulse_start  : time;
  begin
    if rising_edge(clk) then
      pulse_start := now;
      stored_value := d_in;
      d_out <= stored_value;
    else
      assert now = 0 ns or (now - pulse_start) >= 5ns
        report "clk pulse too short: " & time'image(now - pulse_start);
    end if;
  end process store_and_check;
end architecture check_timing;
```

assert-Anweisungen 2

Testbench mit interner Referenz

- Erzeugung der Stimuli und Auswertung in VHDL codiert



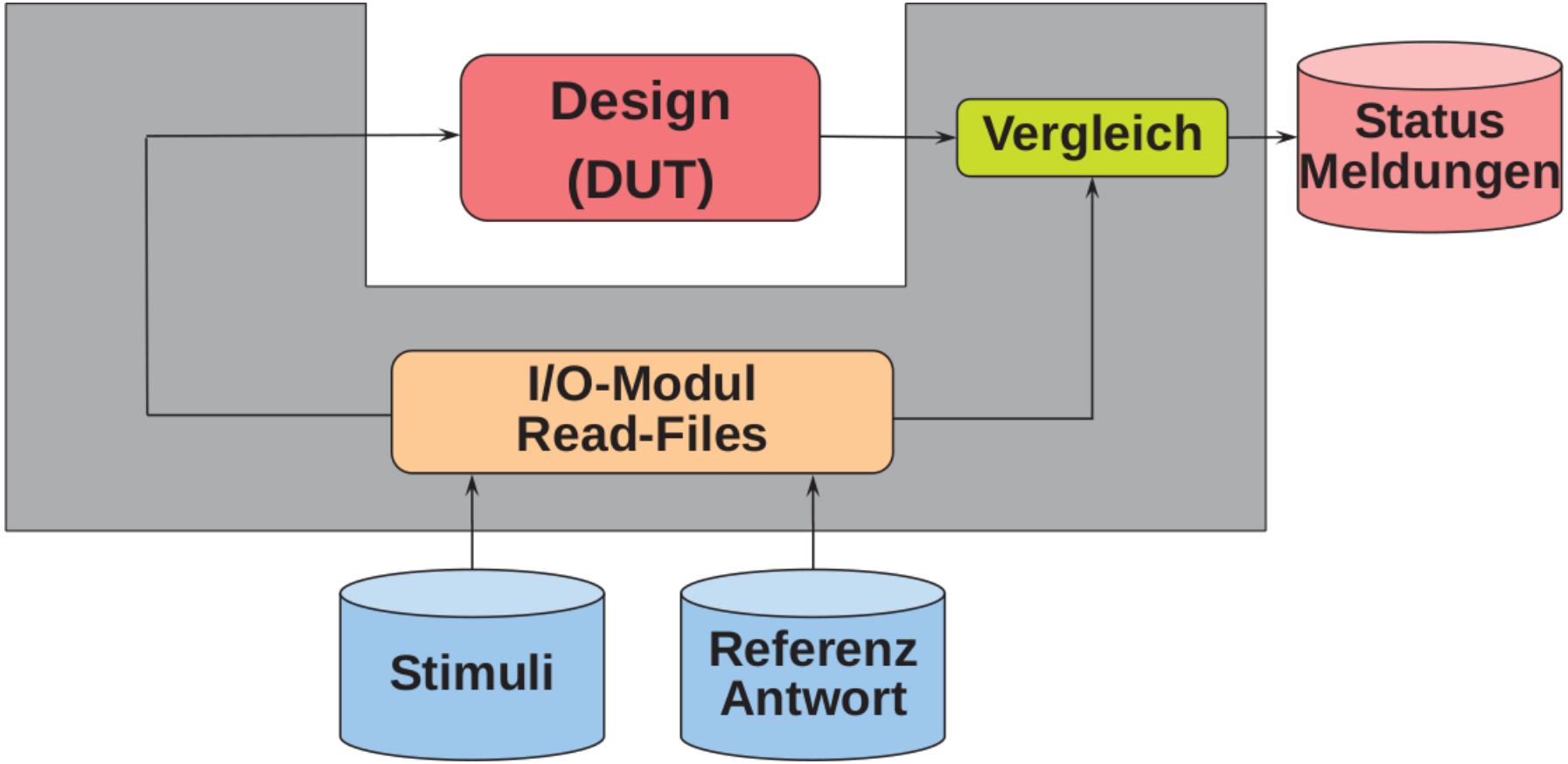
Testbench mit interner Referenz (Beispiel)

```
entity my_tb is
end entity my_tb;
architecture tb of my_tb is
  component adder is
    port (clk  : in  std_logic;
          a, b : in  std_logic_vector(7 downto 0);
          sum  : out std_logic_vector(7 downto 0));
  end component adder;
  signal sys_clk : std_logic := 0;
  signal sys_a, sys_b, sys_sum : std_logic_vector(7 downto 0);
begin
  sys_clk <= not sys_clk after 5 ns; -- Typischer clock-Signal Generator

  STIMULI: process
  begin
    sys_a <= "00001001";              -- Stimuli Zuweisung
    sys_b <= "01001001";
    wait for 11 ns
    assert (sys_sum = "01010010") report "Simulation error" severity failure;
    assert false report "Simulation stop" severity failure;
  end process;

  DUT : adder                         -- Instanzierung DUT
    port map (clk => sys_clk, a => sys_a, n => sys_b, sum => sys_sum);
end architecture tb;
```

Testbench 1

Testbench mit externer Referenz

- Stimuli und Referenzdaten aus Datei gelesen
- Stimuli- und Referenzerzeugung: externe Tools (Script o.ä.)



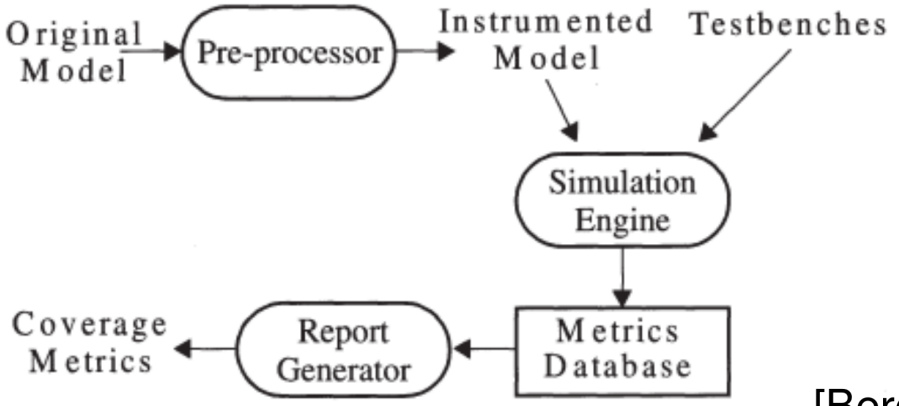
Testbench mit externer Referenz (Beispiel)

```
architecture tb of my_tb is
  component adder is
    ...
  end component adder;
  signal sys_clk : std_logic := 0;
  signal sys_a, sys_b, sys_sum, ref_sum : std_logic_vector(7 downto 0);
  file in_file: TEXT open read_mode is "stimuli.txt";
begin
  sys_clk <= not sys_clk after 5 ns;
  FILEIO: process
    variable in_line: line;
    variable in_str: string(7 downto 0);
  begin
    wait for 1 ns
    while not endfile(in_file) loop
      readline(in_file, in_line);
      read(in_line, in_str);
      sys_a <= to_std_logic_vector(in_str);
      sys_b <= to_std_logic_vector(in_str);
      ref_sum <= to_std_logic_vector(in_str);
      wait for 10 ns
      assert (sys_sum /= ref_sum) report "Simulation error" severity failure;
      assert sys_sum = ref_sum) report "Simulation correct" severity note;
    end loop;
  end process FILEIO;
  DUT : adder port map (clk => sys_clk, a => sys_a, n => sys_b, sum => sys_sum);
end architecture tb;
```

Testbench 2

Coverage Metrics: Code Coverage (I)

- *Code coverage* ist ein Werkzeug, mit dem festgestellt werden kann, welcher Code im DUT ausgeführt wurde und - was noch wichtiger ist - nicht ausgeführt wurde.
- *Code coverage* wird in der Softwareentwicklung schon seit längerer Zeit verwendet.
- Das Problem bei falsch-positiven Tests (d.h. ein fehlerhaftes Design wird für gut gehalten) ist, dass sie genauso aussehen wie ein erfolgreicher Test.



[Bergeron03]

Coverage Metrics: Code Coverage (II)

Statement, *line* und *block coverage* misst, welche Zeilen des Codes von der Simulation ausgeführt wurden.

- Für 100% *statement coverage* muss man verstehen, welche Bedingungen erforderlich sind, um die Ausführung des nicht ausgeführten Codes zu veranlassen.

```
✓   25    PAR_INST : process(D)
    26    begin
    27        PARITY_PROC(X   => D,
✓   28                    PAR => GERADE);
  X 29        case(D(1 downto 0)) is
✓   30                when "00" => B <= "00";
✓   31                when "01" => B <= "00";
✓   32                when "10" => B <= "00";
✓   33                when "11" => B <= "00";
  X 34                when "ZZ" => B <= "00";
✓   35                when others =>
    36        end case;
    37    end process PAR_INST;
```

- Nicht ausgeführter Code ist ein Hinweis darauf, dass eine oder mehrere Testbenches entweder nicht funktional korrekt oder unvollständig sind.

Coverage Metrics: Code Coverage (III)

- Es ist möglich, Code der nicht zur Ausführung vorgesehen war und aus der *Coverage*-Bestimmung auszuschließen (z.B. mit Pragmas)

```
✓  25    PAR_INST : process(D)
   26    begin
   27        PARITY_PROC(X   => D,
✓  28                    PAR => GERADE);
   29        case(D(1 downto 0)) is
✓  30                when "00" => B <= "00";
✓  31                when "01" => B <= "00";
✓  32                when "10" => B <= "00";
✓  33                when "11" => B <= "00";
   34                -- coverage off
E  35                when "ZZ" => B <= "00";
   36                -- coverage on
✓  37                when others =>
   38        end case;
   39    end process PAR_INST;
```

Coverage Metrics: Code Coverage (IV)

- *Conditional/Expression coverage* überprüft die verschiedenen Wege (Bedingungen), auf denen Pfade durch den Code ausgeführt werden.

```
30        if D(1 downto 0) = "00" or D(1 downto 0) = "ZZ" then
31                B <= "00";
32        elsif D(1 downto 0) = "01" or D(1 downto 0) = "10" then
33                B <= "01";
34        else
35                B <= "10";
36        end if;
```

Condition Coverage for:

```
((D(1 DOWNTO 0) = "00") or (D(1 DOWNTO 0) = "ZZ"))
```

FEC Coverage: 1 out of 2 input terms covered = 50.00%

Input Terminal	Covered	Reason	Hint
(D(1 DOWNTO 0) = "00")	Y		
(D(1 DOWNTO 0) = "ZZ")	N	'_1' not hit	Hit '_1'

- Selbst wenn die *statement coverage* 100% beträgt, kann die *conditional coverage* nur bei 50% liegen.

Coverage Metrics: Code Coverage (V)

- *Toggle coverage* gibt einen Bericht darüber, wie oft Signale und Ports während eines Simulationslaufs umgeschaltet werden. Außerdem wird die Aktivität im Entwurf gemessen, z. B. ungenutzte Signale oder Signale, die konstant bleiben oder deren Wert sich nicht ändert.

Toggle Coverage for ALL instances --

Node	1H->0L	0L->1H	0L->Z	Z->0L	1H->Z	Z->H1	ExtMode	"Coverage"
/parity_tb/data(0)	1	1						100.00
/parity_tb/data(1)	1	1						100.00
/parity_tb/data(2)	0	1						50.00
/parity_tb/data(3)	0	0						0.00
/parity_tb/gerade_function	1	1						100.00
/parity_tb/gerade_procedure	1	1						100.00
/parity_tb/paritaet_proc_inst/B(0)	1	1	0*	0*	0*	0*	2-STATE	100.00
/parity_tb/paritaet_proc_inst/B(1)	1	1	0*	0*	0*	0*	2-STATE	100.00

Code Coverage vs. Functional Coverage

- *Code coverage* gibt an, wie gründlich das Design von der Testbench in der Simulation überprüft wird.
  - Sie sagt nichts über die Korrektheit oder Vollständigkeit der Simulation aus.
- *Functional coverage* gibt an, welche interessanten und relevanten Bedingungen überprüft wurden. Sie gibt einen Hinweis auf die Gründlichkeit der Verifikation.
  - Solange einige Signalwerte nicht als ungültig definiert sind, kann sie keine Aussage über die Korrektheit dieser Bedingungen machen.
  - Sie macht keine Aussage über die Vollständigkeit des funktionalen *coverage* Modells.

Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.